

ZERO STRIKE

Jeu de tir tactique multijoueur sur LAN
jusqu'à 40 joueurs simultanés

 **Grand écran**

/display — Phaser 3

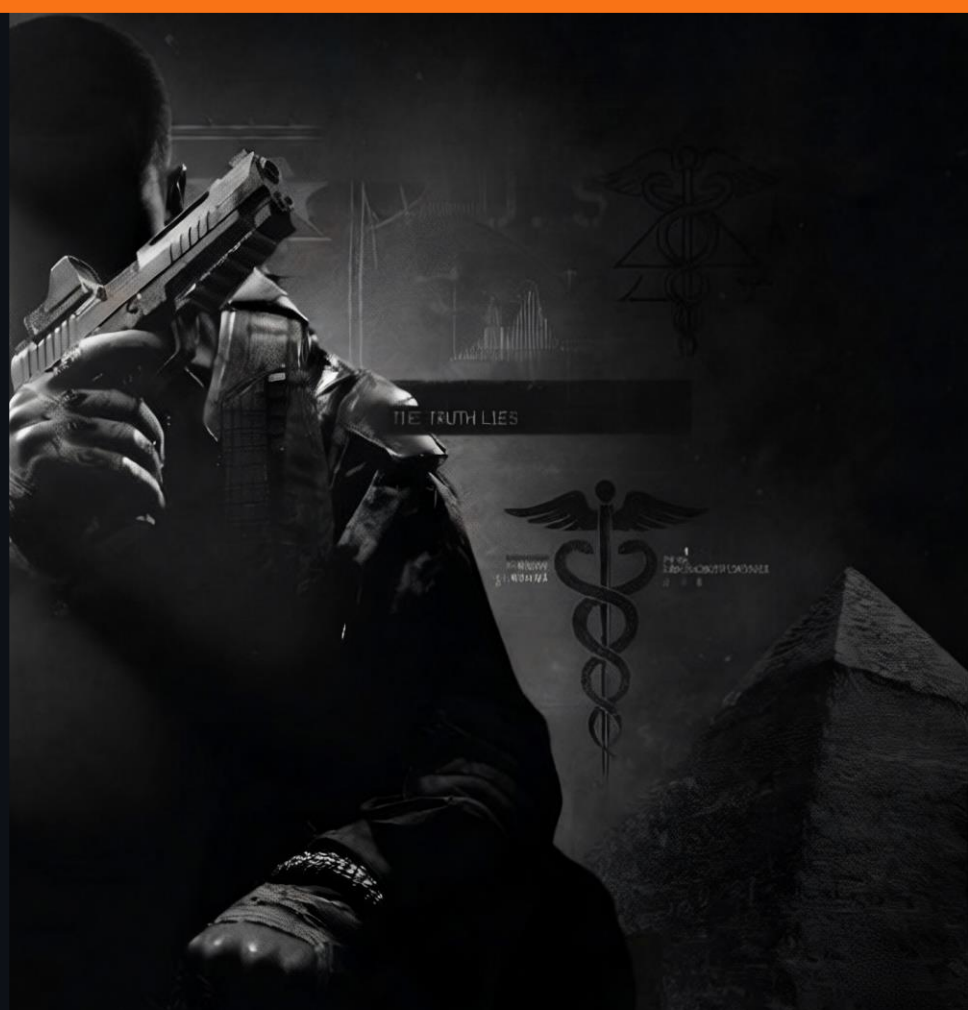
 **Smartphone**

/mobile — joystick, tir, shop

 **Hub /**

orientation joueurs

TP4E-G1 · Ilian El Bouazzaoui Prieur · Mahamat Ibrahim

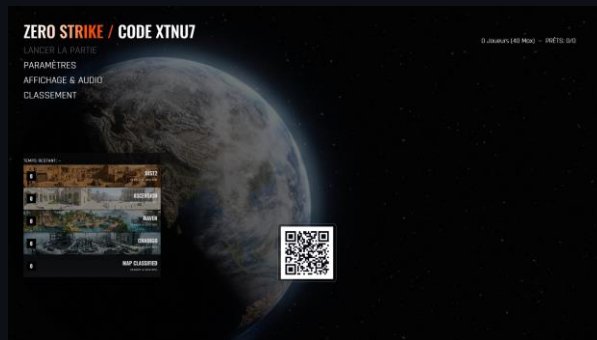


Pitch & Public cible

BLOC 1 — GDD

- **Pitch :**
affrontement tactique temps réel, pensé salle de cours, LAN party, soutenance jury.
- **Objectif joueur :**
gagner la manche (S&D) ou dominer au score (DM).
- **Public :**
du casual au compétitif — 1 serveur + téléphones.
Démon sur instance Render (plan gratuit) en HTTPS, ou LAN en secours

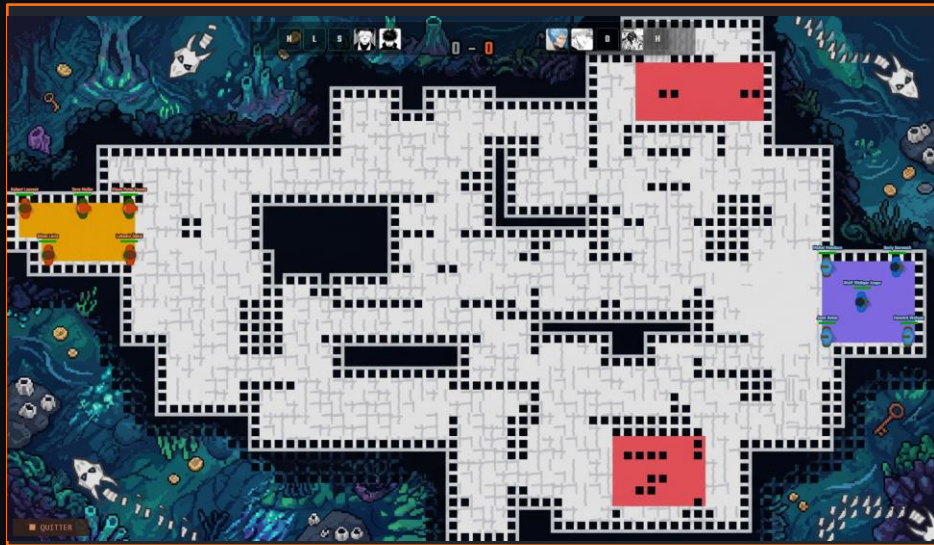
Lobby



Modes de jeu

BLOC 1 — GDD

- **Search & Destroy (S&D) :**
Attaque / défense, plantage / désamorçage.
Sites A / B (+ C selon carte).
Économie entre manches (règles serveur).
- **Deathmatch (DM) :**
Victoire par limite de frags configurable.
- **Lobby :**
Équipe, vote de carte, prêt — lancement par l'hôte (onglet display).



Partie sur la map Acension

Boucle d'une manche

BLOC 1 — GDD

1

LOBBY

QR / URL · pseudo · équipe
vote map · attente hôte

2

BUY PHASE

Armes / héros · budget
10–14 s (selon preset)

3

ACTION

Déplacements · tirs · bombe
(S&D)
~65–105 s (selon preset)

4

FIN DE ROUND

Score · économie
manche suivante

Originalité — Ce qui nous distingue

BLOC 1 — GDD

- **Spectacle :**
Séparation nette grand écran public / manette mobile — peu courant en TP « solo écran ».
- **Serveur autoritaire :**
~60 TPS · état vers display ~30 Hz + dirty · clients envoient des intentions (anti-triche simple).
- **Cartes :**
Tiled (.tmj, 80×45) + repli ASCII — design et robustesse.
- **Produit :**
SQLite (sql.js) · API /api/* · CI + Playwright · Docker · Render (plan gratuit).

MOBILE

Intentions

SERVEUR

~60 TPS

DISPLAY

~30 Hz



Menu Principal

Stack & Accès

BLOC 1 — GDD

Rôle	Technologie
Serveur	Node.js 20+ (ESM), Express, Socket.io v4
Display	Phaser 3, Vite
Mobile	HTML5, Nipple.js, Vite
Web	Hub /, santé /health
Données	sql.js, /api/*
Déploiement	Docker Compose · Render (plan gratuit, render.yaml)



Page d'accueil mobile

Organisation — Partage du travail

BLOC 2 — PLANIFICATION

Ilian El Bouazzaoui Prieur

- Serveur Node.js · architecture domain/app
- Gameplay (boucle, rounds, économie, bombe)
- Socket.io : événements, sync 60 TPS
- GameEngine.js · GameLoopService.js

Mahamat Ibrahim

- Display Phaser 3 : HUD, lobby, scènes
- Client mobile : UX, shop, joystick (Nipple.js)
- Synchronisation état display ~30 Hz
- Intégration cartes Tiled + ASCII fallback

← Transversal : Cartes Tiled · DevOps (Docker, Render) · Tests →

Planning — Jalons

BLOC 2 — PLANIFICATION

J1

HTTP + Socket.io ·
lobby minimal ·
architecture
domain/app

J2

Display + mobile ·
déplacements ·
collisions serveur

J3

Tirs · projectiles · S&D
(bombe, économie) ·
scoreboard

J4

Tiled · presets · polish
UX · Docker / Render ·
stabilisation · stress
test LAN

Problèmes techniques rencontrés

BLOC 2 — PLANIFICATION

- **Build / prod :**
Chemins assets (maps/ vs image Docker / Render) · dépendances front bloquant Vite.
- **Gameplay :**
Spawns sur obstacles · axes joystick · économie : prix / plafonds alignés côté serveur.
- **Réseau / perf :**
Free tier Render (cold start, charge) · logs si surcharge boucle 60 TPS · Phaser : destroy/clear pour éviter accumulation objets graphiques.
- **Mitigation :**
npm run build avant push · npm test · documentation docs/ · audit.

Problèmes humains & Organisation

BLOC 2 — PLANIFICATION

- **Coordination :**
Gestion des branches · intégration sur main · déploiement Render.
- **Arbitrage du temps :**
Gameplay vs UI vs maps — trouver le bon équilibre.
- **Design des maps:**
Créativité et récupération des assets / images Tiled (murs, decors, sols)
- **Solutions :**
Messages de commit clairs.
Pair programming sur zones sensibles (socket + display).
Checklist build + test + démo avant chaque merge.
Carte basique réalisée dans Tiled, avec images générées par l'IA en surcouche.

DÉMO : ce que vous allez voir

Hébergement Render

Démo sur l'instance Render (plan gratuit)
<https://zerostrike.onrender.com>
Réveiller le service avant l'oral (cold start).

Matériel salle

Vidéoprojecteur → navigateur / display
Téléphones → même URL / mobile
(Wi-Fi ou données selon la salle)

Plan B (secours)

PC local : npm start
http://<IP_LAN>:3000/...
Si Render ou réseau bloque.

Déroulé : connexion → vote carte → buy → action (tir, HUD, bombe S&D) → fin de manche

« Nous lançons la partie maintenant. »

Scénario démo — Aide-mémoire

BLOC 3 — DÉMO

Étape	Action
1	Hub → https://zerostrike.onrender.com/ ou /display (hôte) — ou Plan B LAN
2	Chaque joueur → .../mobile (même hôte Render)
3	Vote map → lancement hôte
4	Achat + combat + bombe ou fin timer

✓ Render réveillé · ✓ /health OK · ✓ display + mobiles testés · ✓ Plan B prêt · ✓ Batteries OK